
LemmaPortfolio: Anonymous Technical Supplement

Anonymous Author(s)

Affiliation

Address

email

1 This supplement specifies the benchmark, construction, response transcription, and artifact trail
2 behind the four-page paper. It is self-contained for score reproduction: every score-bearing claim
3 can be traced from the supplied response PDF and episode-key transcription to a public episode, a
4 committed label row, and deterministic scoring code. The accompanying anonymous archive contains
5 those files and the verification commands. It also records what the evidence does *not* establish.

6 1 Task definition

7 For each episode, let C be the displayed candidate set, T the displayed target set, and B the selection
8 budget. The released benchmark fixes $|C| = 16$, $|T| = 8$, and $B = 3$. Candidate and target identifiers
9 are local to an episode; a name such as C04 has no meaning in another episode. Candidate and target
10 declaration names are replaced by these identifiers, but the canonical candidate types and target
11 statements remain visible.

12 For each target t , the label records

$$D(t) = C \cap \text{Consts}(\text{value}(t)),$$

13 where `value` is the elaborated Lean declaration value (including opaque theorem values in the
14 audit) and `Consts` returns the constants occurring directly in its expression tree. The intersection is
15 important: a target may use other declarations that are not among the 16 displayed candidates. No
16 transitive closure is taken.

17 For an exact-budget portfolio $S \subseteq C$, define

$$F(S) = \sum_{t \in T} \mathbf{1}[D(t) \cap S \neq \emptyset].$$

18 The scorer enumerates every $S \subseteq C$ with $|S| = B$ in lexicographic order, computes $F^* =$
19 $\max_S F(S)$, and reconstructs the complete set $\mathcal{A} = \{S : F(S) = F^*\}$. A response is correct
20 precisely when it is a valid size- B subset in $\mathcal{A}$. The stored optimum and stored argmax list are treated
21 as assertions to audit, not as authority: scoring stops if either differs from the fresh enumeration.

22 The three secondary item scores are raw target coverage $F(S)$ (reported in aggregate also as
23 $F(S)/|T|$),

$$\text{normalized coverage} = F(S)/F^*, \quad \text{edge recall} = \frac{\sum_t |D(t) \cap S|}{\sum_t |D(t)|}.$$

24 A missing, malformed, wrong-budget, duplicate-ID, or out-of-pool response gets zero on the primary
25 exact-optimum score and all three secondary scores. Validity and operational status are reported
26 separately.

27 1.1 Exact uniform-random reference

28 There are $\binom{K}{B}$ equally likely portfolios for $K = |C|$. Hence, for one episode,

$$\Pr(S \text{ is optimal}) = \frac{|\mathcal{A}|}{\binom{K}{B}}.$$

For a target with $d_t = |D(t)|$, a random portfolio misses all its displayed dependencies with probability $\binom{K-d_t}{B} / \binom{K}{B}$ (taken as zero when $K - d_t < B$). Linearity of expectation therefore gives

$$\mathbb{E}[F(S)] = \sum_{t \in T} \left(1 - \frac{\binom{K-d_t}{B}}{\binom{K}{B}} \right).$$

We average the item-level optimal probabilities and $\mathbb{E}[F(S)]/F^*$. Since each displayed edge is incident to one candidate, its selection probability and the expected edge recall are both $B/K = 3/16$. No Monte Carlo estimate is needed. A single seeded SHA-256 draw is retained only as a parser and provenance check.

2 Source snapshot and episode construction

2.1 Pinned source

The construction begins from MATHLIB v4.33.0 [The mathlib Community, 2026]. Its pinned Git commit is

db584cd6d46c92f209a44c0f1c829460d327499d.

The extraction consists of 256 Parquet shards from the Mathlib Initiative dependency and type datasets [Mathlib Initiative, 2026a,b]. Before mining, the builder recomputes the name, byte size, and SHA-256 digest of every shard and compares the complete inventory with the earlier source receipt. It refuses a missing, extra, reordered, resized, or changed shard. The generation receipt records the complete inventory rather than relying on a directory-level label.

Score reproduction uses the released episode and label JSONL files and does not require these Parquet shards. The pinned-Lean audit can recompute every displayed edge from the named MATHLIB commit. Repeating the original mining search, however, additionally requires the separately archived 256-shard snapshot; its per-file inventory detects alteration but cannot recover missing bytes.

Eligible declarations have a nonempty, parseable public type, a source module, and a proof-body dependency record. We remove source modules used by any earlier LemmaPortfolio prototype and reject any statement whose whitespace-normalized or leading-binder- α -normalized signature appeared in those prototypes. The same two signatures are used to forbid duplicates within and between the new development and blind splits.

2.2 Cohorts, targets, and masks

Each episode is centered on one source module. Candidate cohorts are drawn from declarations that recur as dependencies elsewhere in the corpus. Targets come from the same module and have at least one edge to the cohort. Stable SHA-256 ranks, fixed in the builder, determine cohort trials, target trials, candidate display order, target display order, and final split order. Human or model responses are not inputs to any of these ranks.

The public row contains only the benchmark ID, local candidate and target IDs, canonical statement strings, budget, task text, and source-project label. The released label row maps local IDs back to source declarations and records every $D(t)$, candidate use counts, F^* , and every optimal portfolio. Before accepting a row, the builder rejects any source theorem name visible in any displayed statement. It also rejects exact or α -equivalent candidate–candidate, candidate–target, and target–target collisions.

2.3 Structural stress filters

Table 1 lists the fixed filters applied before final split selection. They were chosen on development experience and then applied identically to development and blind candidates. A candidate is active if it belongs to at least one $D(t)$; “near-optimal” means coverage $F^* - 1$.

The miner further requires 20 fixed methods to miss the optimum. Eighteen use only public strings: candidate position or length ranks, token-count and connective ranks, aggregate candidate–target text similarity, and three token-Jaccard inferred-incidence widths. Two use the true incidence graph:

Table 1: Episode-level construction requirements.

Requirement	Threshold
Displayed candidates / targets / budget	16/8/3
Active candidates	at least 7
Active candidates outside the union of all optima	at least 4
Targets with two or more displayed direct dependencies	at least 2
Coverage of any singleton candidate	at most 4 of 8
Use count of every member of an optimal portfolio	at least 2
Loss after removing any member of an optimal portfolio	at least 1 target
Distinct portfolios at coverage $F^* - 1$	at least 2
Optimal coverage	at least 6 of 8
Number of accepted optimal portfolios	at most 32
Mean pairwise target token-Jaccard similarity	at most 0.6

frequency rank and greedy maximum coverage. This conditioning is disclosed in the public generation receipt. These methods describe how the challenge set was selected and must not be presented as held-out baselines.

The post-freeze character TF-IDF baseline is different in both timing and implementation. It is specified before the final commitment but generated afterward, reads public statements only, and never enters item admission. It uses character n -grams for $n \in \{3, 4, 5\}$, sublinear term frequency $1 + \ln c$, smoothed inverse document frequency $\ln((1 + N)/(1 + \text{df})) + 1$, fitted within each episode over its 24 candidate and target statements, L2 normalization, and cosine similarity. It enumerates all size-three portfolios to maximize $\sum_t \max_{c \in S} \text{sim}(t, c)$, breaking ties lexicographically. Because both methods use surface text, we still describe it as related to the lexical diagnostic family.

Table 2: Final split summary after the pinned-Lean replacement.

	Development	Blind
Episodes / distinct modules	30 / 30	60 / 60
Top-level domains	13	15
Displayed targets	240	480
Displayed direct edges	386	814
Multi-edge targets	99	226
Mean active candidates	8.37	8.60
$F^* = 6 / F^* = 7$	25 / 5	45 / 15
Accepted optimal portfolios	30	78
Construction-diagnostic hits	0	0

The miner began with 350,599 declarations. After eligibility and prior-split exclusions, 186,680 declarations remained across 7,089 module centers. Stable cohort and target trials produced 71,091 structurally complete incidence drafts. Of these, 165 made all 20 construction diagnostics miss, 121 also passed name masking, and 103 also passed within-episode statement identifiability. One initially selected blind module failed the independent Lean edge audit and was excluded by its module hash; the final row was chosen deterministically from the already ranked admissible superset.

The final blind domain counts are: Algebra 10, Data 7, Ring Theory 7, Analysis 5, Topology 5, Algebraic Geometry 4, Category Theory 4, Combinatorics 3, Number Theory 3, Order 3, Dynamics 2, Geometry 2, Group Theory 2, Linear Algebra 2, and Measure Theory 1. The largest domain therefore contributes 10/60 episodes.

3 Development policy and dataset commitment

Thirty development episodes are partitioned in manifest order into two predeclared, nonoverlapping batches of 15. Batch A supplies the sole model-facing difficulty check: a controlled Codex CLI Sol capture obtained 6/15 exact optima (40%). This single check suggested that the revised format was nontrivial. It is development evidence only, not a consumer-chat test result, an upper bound, or a guarantee that blind Sol performance will be below 60%. Batch B is not used for additional model

calibration, and neither batch is used to choose the ordered-support width. No development item is substituted for a blind item. All development labels are public and excluded from blind prompt directories.

The 60 test episodes are selected by the same structural rules from disjoint modules, with a fixed top-level-domain cap. The source theorems remain public; “hidden” refers to the newly composed episode and its incidence row. On 1 September 2026, before the response document was supplied to the evaluator, the project recorded the public test SHA-256 and label-file commitment shown below.

```
b54293b08bc7ddb7110a50908049922f7cfd86f14cb49a4de4e28353aecc6d8
d17c1fd5c6d87635daf8035fe4a0918d1eaa29708eae37cb797cae08c19ef887
```

The same receipt binds aggregate results from the pinned-Lean edge audit. The model-facing prompts contain the public rows, never the incidence labels.

An earlier protocol anticipated a fully frozen manual panel with per-call timestamps, attestations, and closure receipts. The material actually supplied for this study is one PDF of visible responses, not those richer capture forms. We do not manufacture the missing evidence or inherit claims from the unused protocol. The evaluation therefore supports deterministic score reproduction and a prior dataset commitment, but not independent verification of fresh-chat use, interface settings, call chronology, or the exact serving checkpoints.

4 Pinned-Lean replay audit

The source Parquet graph is convenient for mining but is not accepted as its own verification. A second program builds one Lean input stream in a clean MATHLIB checkout and imports Mathlib. For every displayed source declaration it checks that the constant exists and has the expected declaration kind. For every target it reads `ConstantInfo.value?` with opaque values enabled, obtains `Expr.getUsedConstantsAsSet`, intersects that set with the 16 displayed candidate names, and compares the result exactly with the stored row. It also repeats the cross-statement `Expr.eqv` checks.

The Python wrapper binds the public and label commitments, checks the clean MATHLIB commit, verifies Lean 4.33.0 and its compiler commit, and emits only aggregate counts. This keeps private declaration names out of a public audit log. Negative tests mutate a label edge, each commitment, the source inventory, and the environment; every mutation must fail closed.

Table 3: Aggregate pinned-Lean audit of final bytes.

Check	Development	Blind
Episodes	30/30	60/60
Targets re-extracted exactly	240/240	480/480
Stored / re-extracted displayed edges	386/386	814/814
Changed targets	0	0
Added / removed edges	0/0	0/0
Empty target incidence	0	0
Expression-equivalent statement collisions	0	0

The audit verified clean Mathlib commit `db584cd6d46c92f209a44c0f1c829460d327499d`, Lean 4.33.0 compiler commit `d8b18978322de05a8f3dba51ef03cf5461676c17`, all expected declaration kinds, and exact public/private commitments. A separate static audit re-hashed all 256 Parquet shards and reproduced every final development and blind output byte-for-byte.

5 Response intake and scoring protocol

5.1 Direct rows

The score source is a 15-page PDF entitled *Models and their responses*. Its section headings supply the five visible consumer labels used in the paper. The document contains twelve intended five-episode direct responses per row, except for the two anomalies described below. It contains no

137 account-tier, system-prompt, token-usage, latency, screenshot, or provider-log evidence. Accordingly,
 138 we make no claim about an underlying API checkpoint and do not infer operational facts that are
 139 absent from the document.

140 Transcription removes layout whitespace that the PDF extractor inserts inside identifiers, decodes
 141 complete JSON envelopes, and indexes selections by explicit MLP4B.xxxx keys. The expected key
 142 ranges are D01=0000–0004 through D12=0055–0059. Visual order is irrelevant. Every selection
 143 must contain three distinct in-pool IDs. A missing or malformed whole response block gives all five
 144 episodes zero while retaining them in the denominator. This strict rule avoids silently improving an
 145 output by editorial repair. The canonical machine-readable transcription and source-document hash
 146 are released beside the scorer.

147 GPT contains D01–D11, a second D11 response, and no D12. We retain the first D11 occurrence; the
 148 only changed item in the second version has identical exact and secondary scores, so this convention
 149 does not affect any reported number. Qwen 3.8 contains the D12 keys but omits the envelope’s
 150 outer opening brace. We count that entire block invalid; adding the brace would increase its exact
 151 score from 12/60 to 15/60, but that repaired value is not reported. All other blocks decode and align
 152 uniquely by episode key.

153 5.2 Ordered-support diagnostic

154 Four additional GPT SOL blocks provide a ranked candidate-support list for every target in D04,
 155 D05, D06, and D12. The analysis truncates each list to its first $q = 2$ entries and enumerates all
 156 560 portfolios against the predicted incidence, selecting the lexicographically first maximizer. This
 157 postprocessor uses no gold label. After label release, full and truncated support maps are compared
 158 with the direct-incidence graph using micro precision, recall, and F1. The same 20 episode IDs permit
 159 a descriptive comparison with direct prompting. Because the PDF does not establish fresh-chat use,
 160 counterbalanced chronology, or an absence of intervening feedback, we do not call the difference a
 161 causal prompting effect and report no significance test.

162 6 Complete results

163 6.1 Complete score table

Row	Exact	Valid	Mean $F/8$	Mean F/F^*	Edge recall
SHA–256 deterministic draw	0/60	60/60	2.30/8	37.1%	18.9%
Character TF–IDF	0/60	60/60	3.77/8	60.4%	29.5%
GPT SOL 5.6 Pro	25/60	55/60	4.98/8	79.6%	38.9%
DeepSeek Instant + DeepThink	1/60	60/60	4.20/8	67.1%	34.9%
DeepSeek Expert + DeepThink	7/60	60/60	4.60/8	73.4%	36.9%
Qwen 3.8 Max–Thinking	12/60	55/60	4.32/8	68.9%	34.0%
Qwen 3.7 Plus–Thinking	6/60	60/60	4.05/8	64.6%	31.8%
GPT SOL support $q = 2$	1/20	20/20	4.60/8	74.8%	34.5%

Table 4: Complete fixed-denominator scores. Invalid responses contribute zero to every metric. The analytic uniform-random expectation is 0.14 exact episodes, 2.35/8 mean coverage, 37.6% mean F/F^* , and 18.75% edge recall.

164 For the five consumer rows, item-wise Wilson 95% intervals for exact accuracy are, in table order,
 165 30.1–54.3%, 0.3–8.9%, 5.8–22.2%, 11.8–31.8%, and 4.7–20.1%. Resampling the twelve five-item
 166 response blocks gives descriptive 95% percentile intervals of 30.0–53.3%, 0.0–5.0%, 5.0–18.3%,
 167 8.3–33.3%, and 1.7–20.0%. These intervals summarize sensitivity within this fixed challenge set;
 168 they are not estimates for model families.

169 6.2 Block-level exact scores and response alignment

170 The source document does not always present blocks in numerical order. The alignment
 171 is recovered from the explicit episode keys. GPT supplies D01–D11, then a second D11
 172 variant, and no D12. Both D11 variants score 3/5 and have the same secondary met-
 173 rics; the first is retained as the canonical transcription. DeepSeek Instant and Expert sup-
 174 ply D01–D12. Qwen 3.8 supplies D01,D03,D02,D04,D06,D05,D08,D07,D09,D10,D12,D11;

Row	D01	D02	D03	D04	D05	D06	D07	D08	D09	D10	D11	D12
GPT SOL	2	2	2	2	1	1	4	2	3	3	3	0
DeepSeek Instant	0	0	0	0	0	0	0	0	1	0	0	0
DeepSeek Expert	1	0	0	0	1	0	1	1	0	1	2	0
Qwen 3.8 Max	1	1	1	0	0	0	1	3	3	0	2	0
Qwen 3.7 Plus	0	0	0	0	0	0	1	2	1	0	0	2

Table 5: Exact optima per five-episode direct block. Each row sums to the main-table numerator.

its D12 text is malformed because the outer opening brace is absent. Qwen 3.7 supplies D02,D01,D03,D04,D08,D07,D10,D09,D05,D06,D12,D11. No positional guess is used.

6.3 Structured-support diagnostic

The four support blocks cover D04, D05, D06, and D12. The fixed $q = 2$ postprocessor produces block exact counts 1, 0, 0, 0, compared with direct counts 2, 1, 1, 0 on the same IDs. The corresponding exact-accuracy changes are $-20, -20, -20, 0$ percentage points. Full ranked support has micro precision/recall/F1 of 72.5/75.6/74.0%; after retaining two candidates per target the figures are 76.8/68.7/72.6%. Because the direct D12 response is missing, its secondary-score contrast is dominated by the strict zero and should not be read as a clean prompt ablation.

6.4 Provenance boundary

The public test file and label digest were recorded on 1 September 2026. The score source is the single author-supplied PDF received on 2 September 2026. Their SHA-256 digests, in that order, are:

```
b54293b08bc7ddb7110a50908049922f7cfd86f14cb49a4de4e28353a6cc6d8
d17c1fd5c6d87635daf8035fe4a0918d1eaa29708eae37cb797cae08c19ef887
2f20fbc4b3eb3b87c4b693058fc5744a947de04e19a2eca70f0ea5c0d54dde3f
```

The archive preserves this PDF, the episode-key transcription, and the scoring outputs. It does not contain provider-side request logs, per-chat timestamps, or independently verified deployment screenshots. We therefore describe the rows as externally captured consumer responses with a pre-existing dataset commitment, not as checkpoint-reproducible or cryptographically escrowed runs.

7 Design history: excluded earlier captures

Earlier LemmaPortfolio prototypes are not pooled with this benchmark. They used smaller candidate pools, a two-item budget, one privileged exact pair, and post-release inspection. Their web-interface captures and local quantized-model captures differ in routing, prompting, context allocation, runtime, and item set. In particular, the preserved evidence for an earlier 21/30 development capture does not establish the GPT-4 identity originally associated with it. We retain the number only as design history and make no GPT-4 result claim.

Likewise, an old local row labelled “DeepSeek” sent 15 episodes through one shared reasoning budget and repeatedly fell back after parser failures. It is not comparable to a fresh consumer-web deployment and is not evidence that the DeepSeek family scores two points. We also explored fresh-item local Q4 captures while redesigning the protocol, but they use a different inference surface and do not enter the final panel. No earlier score or response is reused.

8 Interpretation and limitations

The benchmark has four main boundaries.

First, $D(t)$ is historical. A declaration can be logically useful without appearing in the stored proof, and a stored dependency can be replaceable. The task therefore measures dependency compression,

not causal utility. UseFor, for example, studies reusability and contribution to proof likelihood in a self-play setting [Kasriel et al., 2025]; that is a complementary and stronger utility question.

Second, this is a closed pool with uniform costs. Real systems search much larger libraries and may assign different context or retrieval costs to declarations. The coverage objective is monotone submodular, connecting the task to classical maximum-coverage optimization [Nemhauser et al., 1978], but $K = 16$ and $B = 3$ let us enumerate exactly rather than rely on approximation guarantees.

Third, the stress split is conditioned to defeat fixed diagnostics. Its score is useful for comparing declared methods on this challenge distribution, not for estimating the frequency of hard modules in all of MATHLIB. Raw counts are primary. Item-wise Wilson intervals show binomial-scale variation under an exchangeable-item working model, while whole-block bootstrap intervals show sensitivity to resampling the fixed prompt groups. Neither estimates population performance or model-generation variance. The 20-item intervention has only four supplied five-item blocks and supports no significance claim.

Fourth, consumer-chat deployments hide routing, system prompts, and decoding, and the supplied PDF contains no per-call metadata. The rows should not be read as a provider or architecture ranking. There is no human-expert baseline, and one response per item is not an estimate of generation variance. Codex agents assisted benchmark design, evaluation code, response transcription, and manuscript development; the authors checked the scores and remain responsible for every claim.

9 Artifact map and reproduction

The anonymous archive is organized so that score reproduction is distinct from the more expensive reconstruction of the benchmark:

- data/ development and test public rows and released labels;
- responses/ the source PDF, canonical direct and support transcriptions, and the alignment/anomaly ledger;
- results/ fixed baseline selections and machine-readable aggregate, item, and block scores;
- provenance/ the dataset commitment and aggregate construction and pinned-Lean receipts;
- development_evidence/ the 15-item difficulty capture and its decision receipt;
- construction/ the benchmark builder, tests, and pinned-Lean audit wrapper;
- tools/ a standard-library scorer and manifest builder;
- verify_release.py the one-command release verifier; and
- paper/ anonymous sources and the checked anonymous PDFs.

The one-command verifier uses only the Python standard library. It checks the published hashes, reconstructs all optima from the released incidence rows, and recomputes every model, baseline, and development-check score without calling a model, Lean, or a paid service. The archive also supplies the pinned-Lean audit wrapper and tests. Repeating that extraction additionally requires Lean 4.33.0 and the named MATHLIB snapshot; the original mining search requires the separately obtainable 256 Parquet shards. Exact commands and a short path for a nontechnical collaborator are in the archive README.

References

- Timothe Kasriel, Thomas Lu, Devon Ding, Jingxuan He, and Dawn Song. Usefulness-driven learning of formal mathematics. In *NeurIPS Workshop on Mathematical Reasoning and AI*, 2025. URL <https://openreview.net/pdf/023eb56eadd5685c15a6489e0fddd540279b8c45.pdf>.
- Mathlib Initiative. Mathlib constant dependencies. Hugging Face dataset, 2026a. URL <https://huggingface.co/datasets/mathlib-initiative/mathlib-const-dep>. Version 4.33.0; exact 256-shard inventory recorded in the artifact.
- Mathlib Initiative. Mathlib types. Hugging Face dataset, 2026b. URL <https://huggingface.co/datasets/mathlib-initiative/mathlib-types>. Version 4.33.0; exact 256-shard inventory recorded in the artifact.

259 George L. Nemhauser, Laurence A. Wolsey, and Marshall L. Fisher. An analysis of approximations
260 for maximizing submodular set functions—i. *Mathematical Programming*, 14:265–294, 1978. doi:
261 10.1007/BF01588971.

262 The mathlib Community. mathlib4 source snapshot for version 4.33.0. Git repository com-
263 mit, 2026. URL <https://github.com/leanprover-community/mathlib4>. Commit
264 db584cd6d46c92f209a44c0f1c829460d327499d.